Assignment 3 — Data Structures & Algorithms

Structs, Arrays & Linked Lists (cursor-based / array-of-structs implementation)

STRUCTURES SECTION

> Assume the populated **BookShelf** variable is called **BS**, and traversal uses an int index (the assignment's "int pointer, *trav") that follows Genre.first and BookNode.next, where -1 marks the end of a chain.

1. Display the bookName of the 1st book of the YA genre

```
int trav;
trav = YA.first;
printf("%s\n", BS.books[trav].elem.bookName);
```

2. Update the publishing year of the last book of the Fantasy genre

```
int trav;
trav = Fantasy.first;
while(BS.books[trav].next != -1){
    trav = BS.books[trav].next;
}
BS.books[trav].elem.publishingYear = 2024;  // replace with the actual new year
```

3. Interchange the first and last book of the Manga genre

> Swapping the node *contents* (elem) is simpler and safer than re-linking next pointers, since re-linking would also require tracking the second-to-last node and updating Manga.first.

```
int first, last;
Book temp;

first = Manga.first;
last = Manga.first;
while(BS.books[last].next != -1){
    last = BS.books[last].next;
}

temp = BS.books[first].elem;
BS.books[first].elem = BS.books[last].elem;
BS.books[last].elem = temp;
```

CODING SECTION

1. isUnique — return 1 if all books in a genre are unique, else 0

> Assumption: "unique" is judged by bookName. If your instructor meant a different field (or all fields), swap out the strcmp target.

a. Function Header

```
int isUnique(BookShelf *BS, Genre YA);
```

b. Function Call

```
int result = isUnique(&myShelf, YA);
```

c. Code

```
int isUnique(BookShelf *BS, Genre YA){
    int trav1, trav2;
    trav1 = YA.first;
    while(trav1 != -1){
        trav2 = BS->books[trav1].next;
        while(trav2 != -1){
            if(strcmp(BS->books[trav1].elem.bookName,
                      BS->books[trav2].elem.bookName) == 0){
                return 0;                       // found a duplicate
            }
            trav2 = BS->books[trav2].next;
        }
        trav1 = BS->books[trav1].next;
    }
    return 1;                                   // no duplicates found
}
```

d. Simulation

> YA chain: idx3("Divergent") -> idx7("Legend") -> idx1("Divergent") -> -1

trav1	trav2 (inner loop)	comparison	result
3 ("Divergent")	7, 1	"Divergent" vs "Legend" -> no "Divergent" vs "Divergent" -> match	returns 0 immediately

If all names were distinct instead, trav1 would advance 3 → 7 → 1 → -1, every inner loop finishing with no match, and the function would return 1.

2. deleteByPublisher — remove all books with a given publishingHouse

a. Function Header

```
void deleteByPublisher(BookShelf *BS, Genre *Manga, String publisher);
```

b. Function Call

```
deleteByPublisher(&myShelf, &Manga, "Shueisha");
```

c. Code

```
void deleteByPublisher(BookShelf *BS, Genre *Manga, String publisher){
    int trav, prev, temp;
    trav = Manga->first;
    prev = -1;
```

```
    while(trav != -1){
        if(strcmp(BS->books[trav].elem.publishingHouse, publisher) == 0){
            temp = trav;
            trav = BS->books[trav].next;

            if(prev == -1){
                Manga->first = trav;           // deleting the head node
            } else {
                BS->books[prev].next = trav;   // bypass the deleted node
            }

            deallocSpace(BS, temp);
            Manga->count--;
        } else {
            prev = trav;
            trav = BS->books[trav].next;
        }
    }
}
```

d. Simulation

> Manga chain: idx2("Naruto", Shueisha) -> idx5("One Piece", VIZ) -> idx1("Bleach", Shueisha) -> -1 | deleting all "Shueisha" books

trav	prev	match?	action
2 (Naruto, Shueisha)	-1	yes	Manga->first = 5; dealloc idx2; trav = 5
5 (One Piece, VIZ)	-1	no	prev = 5; trav = 1
1 (Bleach, Shueisha)	5	yes	BS->books[5].next = -1; dealloc idx1; trav = -1

Final chain: Manga.first = 5, node 5's next = -1, count reduced by 2 — only "One Piece" remains.

> Note: the handout's deallocSpace is declared void but ends with "return retVal;" where retVal is never declared — that's a typo in the handout, not something to copy.